extension that allows you to run your entire test suite or just a single file of tests directly from the editor.

RSpec

I recommend using the rspec command directly rather than going through any rake tasks that RSpec defines because it's a bit faster. The rspec command can take one or more arguments—files, directories, or file globs. For file globs, all matching files are run:

```
$ rspec spec/models/project_spec.rb spec/models/task_spec.rb
```

RSpec makes it easy to run a single spec. All you need to do is add the line number to the file along with a colon:

```
$ rspec spec/models/task_spec.rb:5
```

RSpec has a less user-friendly syntax that identifies the tests by their place within each nested block. This version is more robust against adding lines to tests and is what some RSpec tools (like bisect) will output:

```
$ rspec spec/models/task_spec.rb[1:1:1]
```

When RSpec's default formatter runs, RSpec will place the file name and line number of each failing test at the end of the output, suitable for easy copy-and-paste back to the command line to run the test in isolation. You'll probably find this quite useful. There is a configuration line in RSpec's helper file to run the specs in the more verbose documentation format if only one file is being executed. It's commented out by default, but you can find it; it's the block that starts with if config.files_to_run.one?.

If you want to run a set of RSpec specs that isn't quite a single test or a set of files, you can use the RSpec metadata system to tag the tests in question:

```
it "should run when I ask for focused tests", :focus do
end

it "should not run when I ask for focused tests" do
end
```

The top spec here has been tagged with the :focus metadata tag, which can also be written more verbosely as focus: true. You can also specify a spec or example group as focused by prefixing its name with f, as in fit, fdescribe, and fcontext.

You can then run all tests tagged as focused by specifying the tag at the command line:

```
$ rspec --tag focus
```